Camera System Documentation

The camera system is implemented in `Camera.cpp` and `Camera.h`, and it's controlled and utilized within `OGL.cpp`. It features a sophisticated dual-mode operation: an automated, cinematic path and a manual free-look mode.

1. Automated Path Mode (Bézier Curve)

This is the default mode when the application starts. The camera smoothly follows a pre-defined curved path through the scene. This is achieved using a cubic Bézier curve.

Defining the Path

In `OGL.cpp`, a 2D array named `controlPoints` defines the shape of this path:

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\OGL.cpp
float controlPoints = {
                                {20.0f,12.0f,105.0f},
                                {5.0f,10.0f,50.0f},
                                {20.0f,12.0f,20.0f},
                                {15.0f,10.0f,-40.0f}
};
```

These four points (P0, P1, P2, P3) are the control points for a cubic Bézier curve.

Generating the Path Points

During initialization in `OGL.cpp`, the application pre-calculates a series of points along this curve and stores them in a `std::vector<float> BezierPoints`. This is done by iterating a parameter `t` from 0.0 to 1.0 and applying the Bézier curve formula:

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\OGL.cpp
    for (float t = 0.0f; t <= 1.0f; t = t + 0.0015f)
    {
        float x = pow((1 - t), 3) * controlPoints + 3 * pow((1 - t), 2) * t *
controlPoints + 3 * (1 - t) * pow(t, 2) * controlPoints + pow(t, 3) *
controlPoints;

        float y = pow((1 - t), 3) * controlPoints + 3 * pow((1 - t), 2) * t *
controlPoints + 3 * (1 - t) * pow(t, 2) * controlPoints + pow(t, 3) *
controlPoints;

        float z = pow((1 - t), 3) * controlPoints + 3 * pow((1 - t), 2) * t *
controlPoints + 3 * (1 - t) * pow(t, 2) * controlPoints + pow(t, 3) *
controlPoints;
```

```
        BezierPoints.push_back(x);
        BezierPoints.push_back(y);
        BezierPoints.push_back(z);
    }
```

Using the Path

The `Camera` object is initialized with a pointer to this `BezierPoints` vector and a `timer` variable. When `m_bUseBezierCamera` is `true`, the `getLookAtMatrix` function uses these points to construct the view matrix.

The `timer` variable acts as an index to step through the `BezierPoints` vector. The camera's position is set to the current point, and its "look at" target is set to the *next* point on the path. This creates the effect of the camera moving and looking along the curve.

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\modules\Camera\Camera.cpp
vmath::mat4 Camera::getLookAtMatrix(void)
{
    // ...
    if (m_bUseBezierCamera)
    {
        if (m_pBezierPoints && m_pTimer)
        {
            // ...
            viewMatrix = vmath::lookat(
                vmath::vec3(BezierPoints[timer], BezierPoints[timer + 1],
BezierPoints[timer + 2]), // Current Position
                vmath::vec3(BezierPoints[(timer + 3)], BezierPoints[(timer + 3) +
1], BezierPoints[(timer + 3) + 2]), // Next Position (Target)
                vmath::vec3(0.0f, 1.0f, 0.0f)); // Up Vector
        }
    }
    // ...
    return(viewMatrix);
}
```

2. Manual Free-Look Mode

The user can switch to a manual "fly-through" camera at any time. In this mode, the camera is controlled by the keyboard and mouse.

Keyboard Input (Movement)

The `WndProc` function in `OGL.cpp` captures `WM_KEYDOWN` and `WM_KEYUP` events for the 'W', 'A', 'S', 'D', 'Q', and 'E' keys. It then calls the corresponding `setMove...` methods on the camera object, which set or unset bitwise flags in the `m_movementFlags` member variable.

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\OGL.cpp
case WM_KEYDOWN:
{
    switch (LOWORD(wParam))
    {
    //...
    case 'W':
    case 'w':
        gpCamera->setMoveForward(true);
        break;
    //...
    }
}
break;
```

Mouse Input (Rotation)

The `rotateOnMouse` function in `Camera.cpp` is called every frame from the `update` function in `OGL.cpp`. It calculates the difference between the current and last mouse positions to determine an offset. This offset is then used to adjust the camera's `m_pitch` (up/down look) and `m_yaw` (left/right look) angles.

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\modules\Camera\Camera.cpp
void Camera::rotateOnMouse(BOOL bRotate)
{
    // ...
    mouseOffsetCoords = mouseCurrentCoords.x - m_lastMouseCoords;
    mouseOffsetCoords = -(mouseCurrentCoords.y - m_lastMouseCoords);
    // ...
    if(bRotate)
    {
        m_yaw   = m_yaw   + mouseOffsetCoords;
        m_pitch = m_pitch + mouseOffsetCoords;
    }
    // ...
}
```

Update Logic

The `Camera::update()` method is the core of the manual mode. It uses trigonometry to calculate a new `m_frontVector` based on the current `m_pitch` and `m_yaw`. From this, it also calculates the `m_rightVector` and `m_upVector`.

```
// c:\RTR-2024\My-Projects\RTR6_TextureGroup\05-FinalProject-
HanumanChalisa\modules\Camera\Camera.cpp
void Camera::update(void)
```

```
{
    // ...
    // calculate front
    float pitchInRadians = vmath::radians(m_pitch);
    float yawInRadians   = vmath::radians(m_yaw);

    m_frontVector = cosf(yawInRadians) * cosf(pitchInRadians);
    m_frontVector = sinf(pitchInRadians);
    m_frontVector = sinf(yawInRadians) * cosf(pitchInRadians);

    m_frontVector = vmath::normalize(m_frontVector);
    // ...
}
```

It then checks the `m_movementFlags` and updates the camera's `m_position` by moving it along these newly calculated vectors.

Toggling and Integration

- **Toggling**: The user can press the **'C'** key to toggle the `m_bUseBezierCamera` boolean, switching between the automated path and manual free-look modes.
- **Integration**: The main `display` loop simply calls `gpCamera->getLookAtMatrix()`. This single function call returns the correct view matrix regardless of which mode is active, neatly abstracting the camera's complexity from the rendering code.